OOP Lab Project Report Guidelines

---

1 Introduction

This section provides a high-level overview of the project.

- **Project Overview:** Describe the purpose of the application, its functionality, and target users.
- **Problem Statement:** Explain the real-world problem your software solves or the process it automates.
- **Objectives:** List 3–4 specific goals of the project (e.g., use of OOP principles, modular design, file handling).

2 System Features

Describe the main functionalities of your system from a user's perspective.

- Adding, updating, deleting records
- Searching or filtering data
- Menu-driven interaction
- File-based storage

3 System Specifications

- **Development Environment:** IDE/editor used
- **Compiler Standard:** C++ version (e.g., C++17)
- **Operating System:** Platform used

4 OOP Features and Technical Implementation

Explain how OOP concepts are used in your project. Do not write only definitions—relate each concept to your code.

- **Classes and Objects:** Main classes and their responsibilities.
- **Encapsulation:** Use of private data and controlled access.
- **Constructors and Destructors:** Object initialization and cleanup.
- **Inheritance:** Class hierarchy and reuse of code.
- **Polymorphism:**
  - Static: Function/operator overloading
  - Dynamic: Virtual functions

- **Abstraction:** Hiding implementation details using headers or abstract classes.
- **File Handling:**
  - Use of `fstream`, `ifstream`, `ofstream`
  - File modes such as `ios::app`, `ios::binary`
  - Reading and writing data
- **Static Members:** Shared variables across objects.
- **Templates:** Generic programming (if used).
- **STL:**
  - Containers (vector, map, etc.)
  - Algorithms (sort, find, etc.)
- **Exception Handling:** Use of try-catch for error handling.

5 System Design (UML)

Students must include a class diagram in this section.

- Draw a clear class diagram showing classes, attributes, and methods.
- Indicate relationships such as inheritance or association.
- The diagram can be hand-drawn (neat) or created using software tools.

6 Key Logic and Implementation

- **Memory Management:** Use of pointers and dynamic allocation.
- **File I/O:** How data is stored and retrieved.
- **Program Flow:** Explanation of the menu-driven system or main logic.

7 Testing and Results

Students must include console output screenshots.

- Provide sample test cases with input and expected output.
- Include different scenarios (valid and invalid inputs).
- Attach clear screenshots of the program execution (menu, operations, file output).

8 Challenges and Debugging

- Describe errors or issues faced during development.
- Explain how those issues were identified and solved.

9 Limitations

- Mention features not implemented.
- Describe any known limitations or inefficiencies.

10 Conclusion

- Summarize how OOP principles improved the design of the system.
- **Future Scope:** Suggest improvements such as GUI, database, or additional features.

11 References

- `https://cplusplus.com`
- Books, tutorials, or other resources used